

# Building a Local “Jarvis”: Quantization, KV-Cache, RAG, and Tool Use on a 16 GB CPU Machine

---

**Author:** Your Name · **Course:** Applied Machine Learning · **Date:** YYYY-MM-DD

IEEE/ACM-style technical report template (target 4–6 pages). Replace the italicized guidance with your own text and drop in the figures/tables that `jarvis plots` and `jarvis analyze` generate. Tables already reference the committed artifacts so you can paste real numbers directly.

---

## Abstract

---

150–200 words. State the goal (a fully local assistant on commodity 16 GB hardware), the methods studied (quantization, KV-cache management, RAG, MCP tool use), the headline findings (e.g., “Q4\_K\_M is the sweet spot at X tok/s and quality Y/3”), and the main limitation vs. cloud LLMs.

**Index Terms** — local LLM, quantization, KV cache, retrieval-augmented generation, function calling, model evaluation.

---

## I. Introduction

---

- Motivation: privacy, cost, offline capability, reproducibility.
- Problem statement: how far can a 7–8B model on a 16 GB CPU machine go?
- Contributions (bullet list):
  1. A reproducible benchmark of 4 quantization levels on 5 capability axes.
  2. A KV-cache / context-length scaling study with q8 cache comparison.
  3. A fully local RAG pipeline and an A/B study vs. parametric-only answers.
  4. An MCP-style web-search tool with function calling and failure analysis.
  5. An automated, category-wise evaluation of 22 prompts.

## II. System Design

---

- **Architecture diagram** (LLM via Ollama ↔ RAG vector store ↔ MCP tool ↔ eval).
- **Model choice rationale:** Qwen 2.5 7B Instruct vs. Llama 3.1 8B; why Q4\_K\_M on 16 GB. Cite memory budget.
- **Software stack:** Ollama, Chroma, `nomic-embed-text`, DuckDuckGo/Brave.
- **Reproducibility:** fixed seed, temperature 0, pinned dependencies.

### III. Part A — Quantization Study

**Method.** Same base model in Q8\_0/Q4\_K\_M/Q3\_K\_M/Q2\_K; 5 standardized prompts (math, code, summarization, factual, reasoning); 200-token completions; metrics = file size, peak RAM, tokens/sec, quality (0–3 rubric in §III-A).

#### A. Quality rubric

Describe the deterministic 0–3 scorer (keyword/structural heuristics).

#### B. Results

**Table I — Quantization trade-offs** (from `measurements.csv`):

| Quant  | Size (GB) | Tokens/s | Peak RSS (MB) | Avg quality /3 |
|--------|-----------|----------|---------------|----------------|
| Q8_0   | ...       | ...      | ...           | ...            |
| Q4_K_M | ...       | ...      | ...           | ...            |
| Q3_K_M | ...       | ...      | ...           | ...            |
| Q2_K   | ...       | ...      | ...           | ...            |

**Figures.** `quant_size_vs_speed.(png/pdf)` , `quant_tradeoff.png` , `quant_quality_by_category.png` .

**Discussion.** Where is the knee of the curve? Which capabilities degrade first under aggressive quantization (typically math/reasoning)?

### IV. Part B — KV-Cache & Context Length

**Method.** Best model from Part A; context lengths 512/2048/8192/16384; measure tokens/sec, TTFT proxy, peak RAM; compare `f16` vs `q8_0` KV cache.

**Table II — Context scaling** (from `kv_cache.csv`).

**Figures.** `kv_context_vs_latency.png` , `kv_context_vs_ram.png` .

**Discussion.** Linear-ish RAM/latency growth; q8 cache savings at long context.

### V. Part C — Retrieval-Augmented Generation

**Corpus.** N arXiv papers (>50 pages). **Pipeline.** token-aware chunking (size/overlap), local embeddings, Chroma cosine index, top-K retrieval.

**Table III — RAG vs. no-RAG** on 5 questions (correctness, sources cited).

**Discussion.** Grounding reduces hallucination; retrieval failure cases.

### VI. Part D — MCP Tool Integration

**Design.** Tool schema, function-calling loop, bounded rounds. **Tasks.**  $\geq 2$  end-to-end web-search tasks. **Successes & failures.** Document at least one failure (skipped tool call / malformed args / stale snippet) and its mitigation.

## VII. Part E — Evaluation

---

**Suite.** 22 prompts: chat (6), RAG (5), tool (5), multi-step (4), adversarial (3).

**Table IV — Category-wise success rate, latency, tokens** (from `eval_results.json`).

**Discussion.** Where does the system fail, and why?

## VIII. Part F — Analysis, Limits, and Future Work

---

- **Honest limits** (grounded in failed cases): quality ceiling, throughput, context cost, tool reliability, retrieval brittleness.
- **Comparison vs. cloud LLMs** (privacy/cost/latency/quality/context table).
- **Two concrete improvements** with 2× RAM or a small GPU.

## IX. Conclusion

---

2–3 sentences: what works today on 16 GB, and the single biggest lever.

## References

---

1. Vaswani et al., “Attention Is All You Need,” 2017.
2. Devlin et al., “BERT,” 2019.
3. Lewis et al., “Retrieval-Augmented Generation,” 2020.
4. Hu et al., “LoRA,” 2021.
5. Brown et al., “Language Models are Few-Shot Learners,” 2020.
6. Ollama, Chroma, Qwen2.5 / Llama 3.1 model cards — add URLs.

## Appendix A — Reproduction

```
bash scripts/install.sh
jarvis run-all          # produces every table/figure referenced above
```

## Appendix B — Hardware used

CPU model, cores, RAM, OS, Ollama version.